OakGPT Phase 3.3: Quiet-Host Middleware Comparison and Replication Gate

OakGPT Research | November 2026

Abstract

A completed quiet-host, matched middleware development comparison dispatched and completed 36/36 missions across synchronous Fast DDS, loopback UDP Fast DDS and Cyclone DDS, with logging enabled and disabled. All 12 sessions passed clock/ROS/TF startup qualification. Cyclone avoided the long startup command tails observed in both Fast DDS configurations. This supports a replication candidate, not a navigation improvement claim. A fresh 24-world held-out schedule is frozen but dispatched no missions because the current host failed its prespecified load gate. Final actual-motion regressions remain unexecuted. Phase 4 remains NO-GO.

I. Architecture and methodology

The command path is Nav2 controller, Python velocity guard, stamped DDS message, C++ ROS/Gazebo bridge, Gazebo transport, native expiry plugin and simulated actuator. The independent C++ authorization boundary, one-second clock freshness, fail-closed 200 ms lease and 30 s navigation deadline are unchanged. Lease expiration latches; no automatic mission resumption is introduced.

Linux CLOCK_BOOTTIME stage differences are computed only within the same WSL kernel/session. DDS realtime, simulation time and Windows monotonic clocks are not subtracted from each other. Sequence/frame IDs join guard publication to native decoding. The upstream controller-to-guard command is unstamped and cannot be uniquely joined. Reported end-to-end latency therefore covers the matched guard-to-native segment; it is not a full controller-to-actuator bound.

Each backend/logging condition ran once in each of two reused development worlds (empty-210 and block-211), three fixed authorized missions per session. Seeded order is reversed in the second world. Every saved pre-session baseline passed all three five-second samples below 20% host CPU and below 0.5 selected external CPU cores. Each startup probe required two continuous seconds of fresh TF and LiDAR. Fresh owned processes and per-session cleanup were confirmed. No failed trial was replaced. Logging-off retains compiled wrappers but disables trace output; it is not an entirely uninstrumented application.

Environment: Windows host, WSL2 Ubuntu 24.04, ROS2 Jazzy, Nav2 1.3.13, Fast DDS 2.14.6/rmw 8.4.4, Cyclone DDS 0.10.5/rmw 2.2.4, glibc 2.39. This is software-rendered simulation; there is no new RTX 2060 training or model inference measurement. Exact installed hashes are in the freeze artifact [2].

II. Timing attribution and negative results

The earlier 332.892270 ms pthread_mutex_lock wait was mapped by stack and installed disassembly to the Fast DDS participant m_send_resources_mutex_. Its original owner was not captured. A separate later ownership trace recorded a flow-controller network-delivery thread holding the watched mutex for 63.165420 ms; this does not identify the original owner. The installed ordinary mutex acquisition is not bounded by the writer reliability max_blocking_time. Historical equivalence to the 337 ms event remains unproven.

In the quiet comparison, current Fast DDS maxima spent about 319-325 ms in guard publication. In UDP empty-210, guard publication returned quickly while DDS take was approximately 315 ms after publication began. UDP therefore did not eliminate downstream delivery delay. All four Fast DDS/UDP worst matched commands were neutral startup zeros. Cyclone's session maximum matched latencies were 1.375 and 2.370 ms. These are maxima of observed sessions, not real-time guarantees.

Previously tested asynchronous publication reduced maximum API blocking from 321.779 to 109.086 ms, but maximum guard-to-native delay was 322.157 versus 329.688 ms. It was not selected. A no-motor stopped-reader test did not reproduce the long wait. The earlier two development cohorts retained clock, TF, startup and runtime failures (Fig. 2); they are confounded by harness remediation and host load and are not pooled with the quiet cohort as independent efficacy trials.

III. Quiet-host results and logging overhead

cyclone-on block-211: p95 0.718, p99 1.060, maximum 1.375 ms; guard API maximum 0.888 ms.

cyclone-on empty-210: p95 0.704, p99 0.859, maximum 2.370 ms; guard API maximum 0.650 ms.

fast_default-on block-211: p95 0.696, p99 0.868, maximum 318.981 ms; guard API maximum 318.598 ms.

fast_default-on empty-210: p95 0.732, p99 1.467, maximum 325.464 ms; guard API maximum 324.620 ms.

fast_udp-on block-211: p95 0.726, p99 0.944, maximum 316.623 ms; guard API maximum 316.261 ms.

fast_udp-on empty-210: p95 0.868, p99 36.208, maximum 314.946 ms; guard API maximum 3.676 ms.

All three backends completed 12/12 planned missions each across logging modes. There were zero startup failures. All six logging-on sessions had every required stage, zero reported trace errors/drops, actual native physics contact samples and positive floor-contact records. No robot/environment pair was observed. No traced command gap above 200 ms following a nonzero command and no native expiry latch was observed. Logging-off latency, mutex, expiry and contact counts are unavailable, not zero.

Fast DDS empty/block maximum watched mutex waits were approximately 324/327 ms; UDP approximately 329/996 ms. These mutex observations span middleware threads and are not interchangeable with guard publication maxima. Cyclone does not execute the traced Fast DDS lock path; absent samples are not a zero latency measurement.

Mean mission wall durations with logging on/off (seconds; six missions per mode, descriptive only): fast_default: 11.618/10.718; fast_udp: 10.132/10.831; cyclone: 10.481/11.363. Order, process scheduling and navigation variability confound attribution. No

statistical logging penalty or efficacy claim is justified from these two reused world clusters.

Separate 20,000-call microbenchmarks recorded native enabled median 1.650 us and p99 5.317 us; Python enabled median 5.500 us and p99 29.700 us. Disabled medians were 0 ns native (timer resolution) and 275 ns Python. These local trace-call costs exclude full application storage, contention and scheduler effects.

IV. Frozen replication and verification status

Cyclone was selected from the completed development study solely for its lower observed tails and successful dispatch. The source commit and installed artifact hashes were frozen before a new random master seed generated eight start-seed clusters spanning empty, block and wall families (24 worlds). The immutable paired schedule balances 12 candidate-first and 12 baseline-first worlds. The comparison preserves Phase 3.2 Nav2 parameters with common passive instrumentation. It is not the byte-identical historical Phase 3.2 binary.

No held-out mission was dispatched: the runner stopped at its load gate. Fresh host samples v13 measured 34.2-44.4% CPU, v14 25.5-35.4%, and the actual runner preflight 21.7-29.5%, exceeding the unchanged 20% ceiling. Selected previously identified applications were below 0.5 cores, but total load still failed. Local process diagnostics are excluded from Git. The frozen worlds, schedule, failed preflight and resume command remain preserved. Final ten-case actual-motion safety regressions are not executed and cannot be scored as passes.

Completed software verification: 91 Python tests passed, two hardware tests skipped; Ruff lint/format and strict mypy passed. Fresh C++ release and ASan/UBSan tests passed (10,031 validator checks and 15,036 lease-policy checks per mode). All 60 frozen C++ oracle scenarios passed. Actual C++ mailbox integration accepted a fresh mission and rejected stale/future missions. Unit tests cover stale/replayed/corrupted local clock data under the original one-second bound. ROS packages passed 10 tests with no failures/errors/skips; two installed expiry tests and 20,000 native clock-semantics checks passed. Packaging and archive/CI status are recorded separately in verification documentation.

V. Limitations, decision and next work

Supplementary saved resource analysis reports maximum nominal-100-ms sampling gaps of 0.958-2.264 s across sessions. Linux CPU-pressure some/avg10 peaks ranged from 2.54 to 4.21%. These are not Windows utilization measurements. Only observed main-thread schedstat deltas are available; middleware workers are incompletely covered. Near-command observations cannot assign causal mutex ownership or prove uninterrupted low contention.

Two reused development worlds do not estimate generalization. Commands and repeated answers are not independent experimental units. Session p95/p99 values are descriptive; no confidence interval across commands is presented. The prespecified paired bootstrap uses eight independent world/start clusters, 10,000 resamples and seed 32, but cannot be computed without the held-out trials. Contact sample coverage is finite; absent observed contact does not establish collision impossibility. Localization, motion settling and expiration under process failure require final independent ground-truth measurements.

Phase 4 decision: NO-GO pending evidence. The unchanged criteria require at least 65/72 candidate missions completed, zero nominal native expiry latches, zero observed robot/environment contact missions, at least 95% of completed truth arrivals within 0.35 m (missing truth does not pass), eligible independent-motion settling within 0.5 simulated seconds for every preregistered fault, and all software/hosted checks passing. None of these held-out or fault requirements is waived by the development result.

Next: obtain a measured quiet host, resume the existing frozen paired schedule without replacing trials, execute all ten actual-motion faults with confirmed nonzero preconditions and native stale/replay ingress, then aggregate cluster uncertainty and issue a final decision. If a criterion fails, preserve it and report bounded limitations. Stop robotics feature expansion at this simulation demonstrator. LoRA remains deferred; physical deployment and physical stopping guarantees are outside scope.

References and evidence

[1] reports/phase33/backend-development-quiet-v1-analysis-v2.json and byte-preserved raw/manifest.json archives. [2] configs/robotics/phase33_freeze_v1.json; configs/robotics/phase33_protocol.json. [3] runs/phase33/held-out-worlds-v1/generation.json and held-out-replication-v1/schedule.json, archived with blocked replication evidence. [4] reports/phase33/verification-v2/raw/manifest.json. [5] Historical attribution, failed development cohorts and remediation archives under reports/phase33; historical Phase 1-3.2 evidence is unchanged.

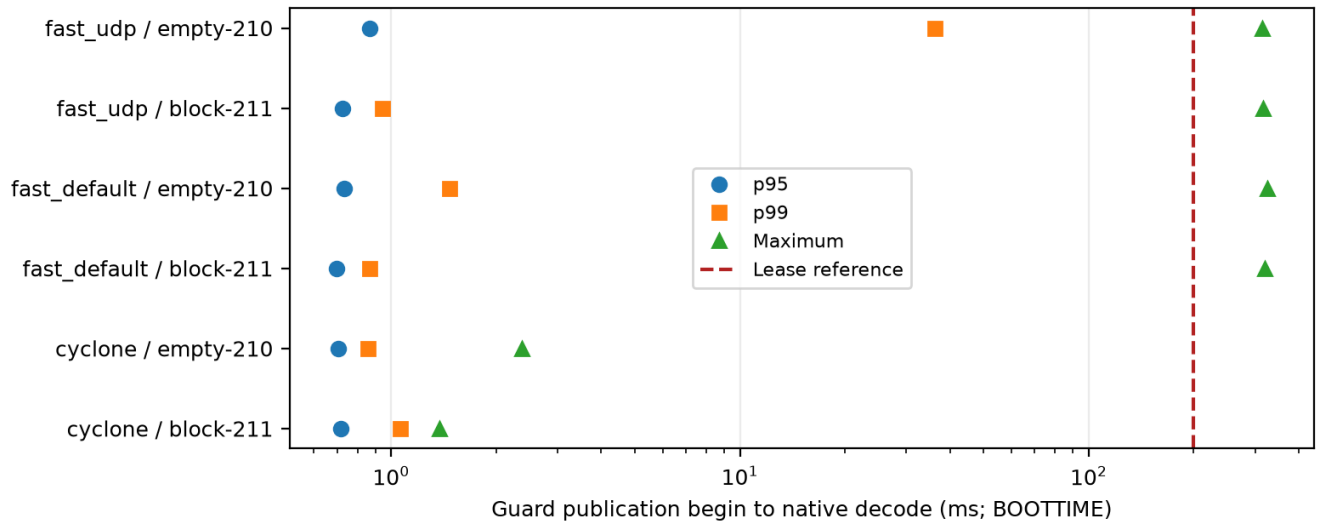


Fig. 1. Session timing tails in the completed quiet cohort. Logging-off latency is unavailable. The lease line is a reference, not a full-path deadline.

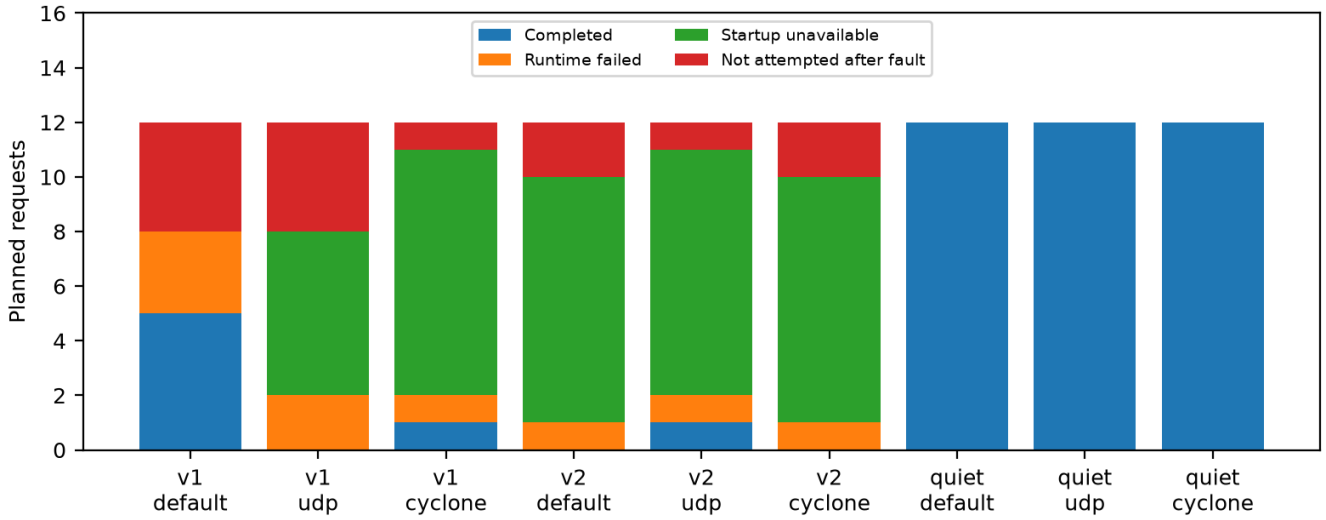


Fig. 2. Historical failed/unavailable cohorts and the completed quiet cohort, kept separate. Between-cohort changes include harness remediation and host conditions; differences are not a causal middleware effect.